UI / UX

Candex's interface is an **Ionic 8 + Angular 20** mobile app. This handbook is the design and interaction side; for how the front-end code is structured, see Front-end.

Design tokens

Colours, typography, and surfaces are defined once as **Ionic CSS variables** and reused everywhere, so the brand stays consistent across every screen.

primary
#00B1FF
Candex blue — primary actions, accents

secondary
#ffffff
White

tertiary
#033C44
Deep teal — secondary emphasis

success
#2dd36f
Success state

warning
#ffc409
Warning state

danger
#eb445a
Danger / error state

Each colour ships with its `-shade`, `-tint`, and `-contrast` variants, so text always meets its background. Typography is **Rubik**, self-hosted from `assets/fonts` (Regular, Medium, Bold, and the rest of the family) and wired as `--ion-font-family`, so the app renders identically offline and across platforms.

Typography

The app's typeface is **Rubik**, self-hosted from `assets/fonts` and set as `--ion-font-family`. Three weights are loaded and carry the whole hierarchy: **Regular (400)** for body text, **Medium (500)** for emphasis and labels, and **Bold (700)** for headings and strong text.

Text sizing follows **Ionic's built-in typography** (the app imports `typography.css`) rather than a bespoke scale: a **16px (1rem)** base, with headings stepping down from `<h1>` to `<h6>` on Ionic's default scale. In short, the app owns the *family*; Ionic owns the *sizes* — which keeps text consistent and respects the platform's accessibility (dynamic type) settings.

Styling conventions

Styles are written in SCSS and follow a few strict rules, so the codebase stays uniform:

- **Cascade, always** — styles are written as nested SCSS that leans on the cascade, not flat, repeated selectors.
- **A fixed spacing scale** — margins, paddings, gaps, and offsets use only **2, 4, 8, 16, 32** px. `12` and `24` are allowed, but only in rare, specific cases.
- **Global variables for shared values** — colours, fonts, and text sizes always come from global variables (the design tokens), never hard-coded literals.
- **No floating CSS** — no inline styles in templates and nothing styled outside a `.scss` file; every rule lives in SCSS.
- **Consistency above all** — the same thing is done the same way everywhere.

Theming — Light, Dark, System

The app offers three themes through a `ThemeService`: **Light**, **Dark**, and **System** (which follows the device setting). The choice is applied by setting a `theme` attribute on `<body>`, and the token blocks `body[theme=light]` / `body[theme=dark]` swap the palette underneath — every component reacts automatically because it reads the same variables.

Component library

On top of Ionic's own components (buttons, cards, lists, modals, tabs) sit a set of **shared components** that carry the app's specific patterns:

- **Content** — `drink-card` renders a can consistently across the Dex and lists.
- **Feedback** — `banner` and `error-banner` surface inline messages; `form-errors` renders field-level validation under inputs.

- **Focused tasks (modals)** — `avatar-cropper-modal`, `fullscreen-image-modal`, `report-problem-modal`, and the Google account flows (`google-account-modal`, `google-link-prompt-modal`, `google-photo-choice-modal`) keep one task in view at a time.
- **Auth** — `social-auth-buttons` provides the social sign-in entry points.

Icons come from **Ionicons**, the set bundled with Ionic.

Navigation

The app runs on a **three-tab shell** (`tabs.page`):

- **Dex** — the collection, and the app's default landing tab.
- **Search** — a tap action that opens search rather than a static tab page.
- **Profile** — the user's account and settings.

Pages are **lazy-loaded standalone components** (`loadComponent`), so each screen loads on demand. Outside the tabs, an **auth stack** carries the sign-in journey — `welcome`, `register`, `login`, `verify-email`, `two-factor`, and `complete-profile` — and dedicated routes handle detail and settings views (`drink-details/:drinkId`, `settings`). Authentication itself is covered in Security.

Interaction & content conventions

A few patterns repeat throughout:

- **Modals for focused work** — cropping an avatar, viewing an image fullscreen, linking a Google account, or reporting a problem each happen in a modal, keeping the underlying screen intact.
- **Inline, field-level validation** — forms show errors next to the offending field via `form-errors`, not as disruptive alerts.
- **Banners for status** — transient and error messages appear as banners rather than blocking dialogs.

Internationalization

The UI is fully translated with **ngx-translate**: strings are externalized to `en.json` and `fr.json` and rendered through the `| translate` pipe (`tabs.dex`, `tabs.search`, …), so English and French stay in step and new languages are a matter of adding a locale file.

Where to go next

- The code behind these screens → Front-end
- What the app is for → Understanding Candex
- Sign-in and permissions → Security